

Session 15 Worksheet: Derivatives & Gradients

Understanding Gradient Descent Through Interactive Visualization and Analysis

Machine Learning Course - Deep Dive Edition

May 11, 2025

1 Learning Objectives

Concept

By the end of this comprehensive session, you will:

1. Understand the fundamental relationship between derivatives and gradients
2. Visualize how gradient descent navigates a cost function landscape
3. Implement gradient descent from scratch with detailed parameter tracking
4. Analyze convergence behavior through iteration-by-iteration examination
5. Master the critical relationship between learning rates and convergence patterns
6. Develop intuition for optimization landscapes through hands-on experimentation

2 Background Theory: The Mathematical Foundation

Key Insight

The Heart of Machine Learning Optimization

In machine learning, derivatives represent how a function changes with respect to its inputs. Think of them as a GPS system for optimization: they tell us which direction to move to reduce our model's error. The gradient—a vector of partial derivatives—points in the direction of the steepest increase. By moving in the opposite direction, we perform gradient descent to find the minimum.

This isn't just mathematical abstraction. Every time a neural network learns to recognize images, translate languages, or play games, it's using these principles scaled to thousands or millions of parameters simultaneously.

2.1 Core Mathematical Concepts

Concept

Three Fundamental Ideas

1. **Derivatives:** Measure instantaneous rate of change. For our cost function $f(x) = (x - 3)^2 + 4$, the derivative is $f'(x) = 2(x - 3)$.
2. **Gradient:** A vector containing all partial derivatives. In our one-dimensional case, it's simply the derivative. In higher dimensions, it becomes the compass for multi-dimensional optimization.
3. **Gradient Descent:** An iterative algorithm using gradients to find function minima. Imagine a ball rolling down a hill—it naturally follows the steepest downward path.

3 Step-by-Step Implementation and Analysis

3.1 Step 1: Understanding Our Error Landscape

Let's define our quadratic cost function that will serve as our optimization playground:

$$f(x) = (x - 3)^2 + 1 \tag{1}$$

Important Note**Why This Specific Function?**

This cost function serves as a perfect teaching tool because:

- It creates a perfect bowl shape with a single global minimum
- The minimum is at $x = 3$ with a cost value of 1
- Its derivative is simple: $f'(x) = 2(x - 3)$
- It mirrors the behavior of more complex cost functions in machine learning
- We can visualize and understand every aspect of its behavior

```

1 def cost_function(x):
2     """
3     A simple quadratic cost function: f(x) = (x - 3)^2 + 1
4
5     This represents our error landscape:
6     - Minimum at x = 3 (where the cost is 1)
7     - Bowl-shaped, encouraging movement toward the center
8     - Smooth and differentiable everywhere
9     - Perfect for understanding optimization dynamics
10    """
11    return (x - 3)**2 + 1
12
13 # Let's immediately test our function
14 print("Cost at x=0:", cost_function(0)) # Far from minimum
15 print("Cost at x=3:", cost_function(3)) # At the minimum
16 print("Cost at x=6:", cost_function(6)) # Opposite side

```

Listing 1: Defining Our Cost Function - The Foundation of Optimization

3.2 Step 2: Computing the Gradient - Our Optimization Compass

The derivative tells us exactly which direction reduces the cost:

$$f'(x) = 2(x - 3) \quad (2)$$

Key Insight

Interpreting the Gradient

The gradient acts as our directional guide:

- When $x < 3$: Gradient is negative $\Rightarrow$ function slopes downward to the right $\Rightarrow$ move right
- When $x > 3$: Gradient is positive $\Rightarrow$ function slopes upward to the right $\Rightarrow$ move left
- When $x = 3$: Gradient is zero $\Rightarrow$ we've found the minimum!

This automatic direction-finding is what makes gradient descent so powerful.

```

1 def gradient(x):
2     """
3     The derivative of our cost function:  $f'(x) = 2(x - 3)$ 
4
5     The gradient tells us:
6     - The direction of steepest ascent (positive = right, negative =
7       left)
8     - The magnitude of the slope (how steep the hill is)
9     - Whether we're at a critical point (gradient = 0)
10    """
11    return 2 * (x - 3)
12
13 # Let's examine the gradient at different points
14 points = [0, 1, 2, 3, 4, 5, 6]
15 for x in points:
16     grad = gradient(x)
17     direction = "right" if grad < 0 else "left" if grad > 0 else "
        nowhere (minimum!)"
18     print(f"At x={x}: gradient={grad:6.2f} -> move {direction}")

```

Listing 2: Computing and Understanding the Gradient

3.3 Step 3: Comprehensive Visualization Framework

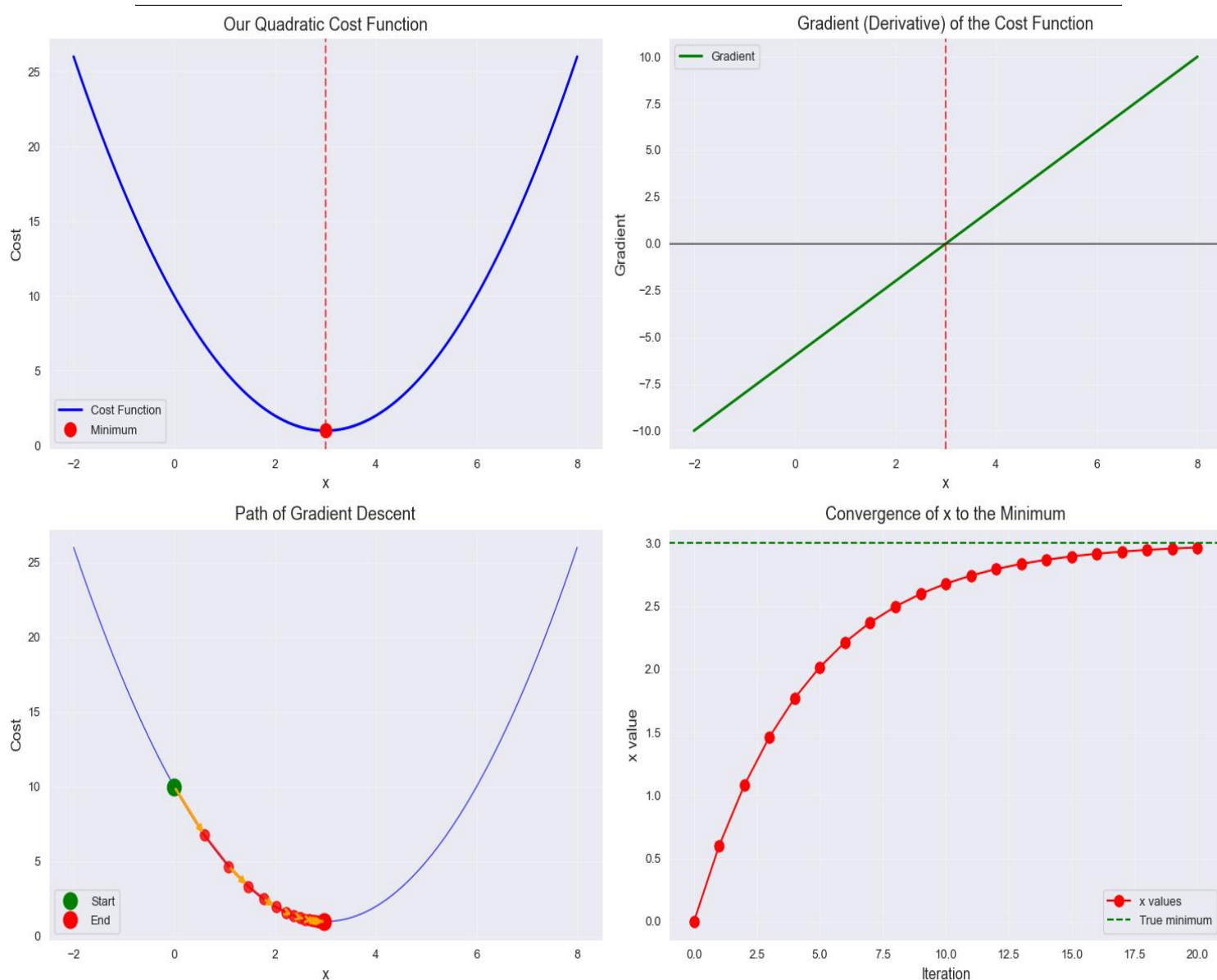


Figure 1: Complete visualization of gradient descent dynamics. Top row: Cost function landscape (left) and gradient field (right) showing the mathematical foundation. Bottom row: Actual descent path (left) with orange arrows showing movement direction, and convergence analysis (right) plotting how x -values approach the minimum over iterations.

Results Analysis

Visual Analysis - What Each Panel Reveals

1. **Top Left - Cost Function:** The blue parabola represents our error landscape. The red dot marks the global minimum at $x=3$.
2. **Top Right - Gradient Function:** The green line shows the slope at every point. Notice it's negative on the left of $x=3$ and positive on the right.
3. **Bottom Left - Descent Path:** Red dots trace our algorithm's journey from start (green) to end (red). Orange arrows show movement direction.
4. **Bottom Right - Convergence:** The algorithm starts at $x=0$ and asymptotically approaches $x=3$, never quite reaching it but getting arbitrarily close.

3.4 Step 4: The Gradient Descent Algorithm - Detailed Implementation

Here's the complete implementation with extensive tracking and analysis:

```

1 def gradient_descent(starting_point, learning_rate, num_iterations):
2     """
3     Comprehensive gradient descent implementation with detailed
4         tracking
5
6     This function doesn't just optimize---it records every step of the
7         journey
8         for deep analysis of the algorithm's behavior.
9     """
10    # Initialize tracking arrays
11    x_history = [starting_point]
12    cost_history = [cost_function(starting_point)]
13    gradient_history = [gradient(starting_point)]
14    step_sizes = [] # Track actual step sizes taken
15
16    current_x = starting_point
17
18    print(f"Starting gradient descent from x={starting_point}")
19    print(f"Learning rate: {learning_rate}")
20    print(f"Target: Find minimum near x=3")
21    print("-" * 50)
22
23    # Perform gradient descent
24    for i in range(num_iterations):
25        # Calculate gradient at current position
26        grad = gradient(current_x)
27
28        # Calculate step size
29        step_size = learning_rate * abs(grad)
30        step_sizes.append(step_size)
31
32        # Take the step (negative of gradient for descent)
33        current_x = current_x - learning_rate * grad

```

```

33     # Record our progress
34     x_history.append(current_x)
35     cost_history.append(cost_function(current_x))
36     gradient_history.append(gradient(current_x))
37
38     # Print detailed information for first few and last few
39     # iterations
40     if i < 5 or i >= num_iterations - 3:
41         cost_val = cost_function(current_x)
42         print(f"Iteration_{i:2d}: x={current_x:8.6f}, cost={
43             cost_val:8.6f}, gradient={grad:8.6f}")
44     elif i == 5:
45         print("... (intermediate iterations) ...")
46
47     print("-" * 50)
48     print(f"Final position: x={current_x:.8f}")
49     print(f"Error from true minimum: {abs(3 - current_x):.8f}")
50     print(f"Final cost: {cost_history[-1]:.8f}")
51     print(f"Final gradient: {gradient_history[-1]:.8f}")
52
53     return x_history, cost_history, gradient_history, step_sizes

```

Listing 3: Complete Gradient Descent with Detailed Tracking

3.5 Step 5: Execution and Detailed Results Analysis

Let's run our gradient descent and analyze the results:

```

1  # Run gradient descent with specific parameters
2  starting_point = 0
3  learning_rate = 0.1
4  num_iterations = 20
5
6  # Execute the algorithm
7  x_hist, cost_hist, grad_hist, steps = gradient_descent(
8      starting_point, learning_rate, num_iterations
9  )
10
11 # Create detailed results table
12 print("\nDetailed Iteration-by-Iteration Analysis:")
13 print("-" * 80)
14 print(f"{'Iter':^6} | {'x_value':^12} | {'Cost':^12} | {'Gradient':^12} |
15       {'StepSize':^12}")
16 print("-" * 80)
17
18 for i in range(min(10, len(x_hist))):
19     if i < len(steps):
20         print(f"{i:^6d} | {x_hist[i]:^12.8f} | {cost_hist[i]:^12.8f} |
21               {grad_hist[i]:^12.8f} | {steps[i]:^12.8f}")
22     else:
23         print(f"{i:^6d} | {x_hist[i]:^12.8f} | {cost_hist[i]:^12.8f} |
24               {grad_hist[i]:^12.8f} | {'N/A':^12}")
25
26 if len(x_hist) > 11:
27     print("... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ...")
28     i = len(x_hist) - 1

```

```
26     print(f"{i:^6d} | {x_hist[i]:^12.8f} | {cost_hist[i]:^12.8f} | {  
27           grad_hist[i]:^12.8f} | {'N/A':^12}")  
28  
29 print("\nConvergence Analysis:")  
30 print(f"- Distance to minimum decreased from {abs(3-x_hist[0]):.6f} to  
31       {abs(3-x_hist[-1]):.6f}")  
30 print(f"- Cost reduced from {cost_hist[0]:.6f} to {cost_hist[-1]:.6f}")  
31 print(f"- Gradient magnitude decreased from {abs(grad_hist[0]):.6f} to  
       {abs(grad_hist[-1]):.6f}")
```

Listing 4: Running Gradient Descent with Comprehensive Analysis

4 Deep Dive: Analyzing Gradient Descent Behavior

4.1 Convergence Patterns and Exponential Decay

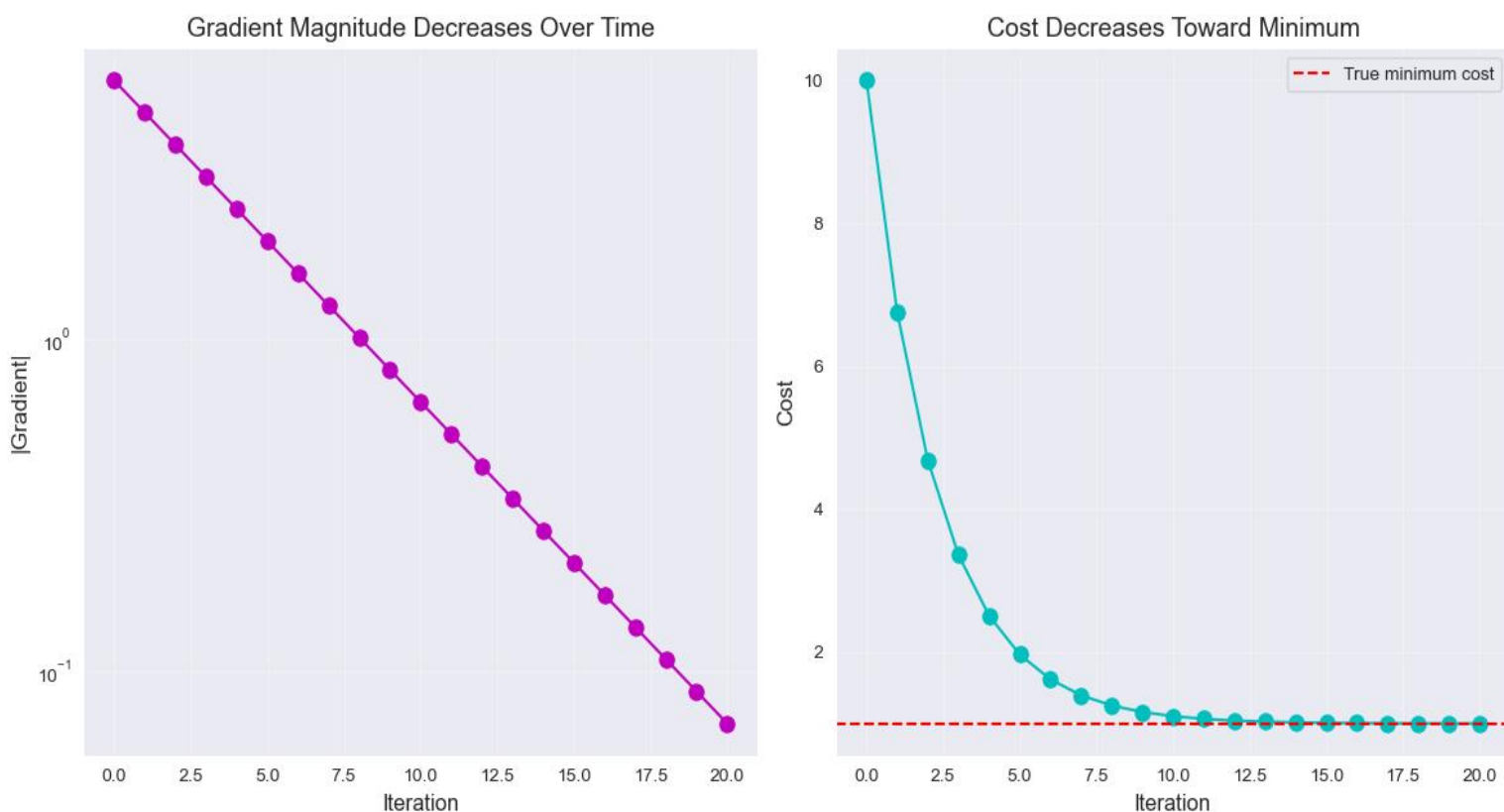


Figure 2: Quantitative analysis of convergence behavior. Left: Gradient magnitude over iterations on logarithmic scale, revealing exponential decay. Right: Cost function value approaching the minimum asymptotically. The red dashed line marks the true minimum cost.

Key Insight

Understanding Exponential Convergence

The logarithmic scale in the left panel reveals a crucial insight: gradient descent exhibits exponential convergence! This means:

- The error decreases by a constant multiplicative factor each iteration
- We can predict convergence time based on this rate
- The algorithm naturally "brakes" as it approaches the minimum
- This pattern holds for many optimization problems in machine learning

Mathematically, if the gradient at iteration k is g_k , then $g_{k+1} \approx (1 - 2\alpha)g_k$ for our function, where α is the learning rate.

5 Critical Analysis: Reflection Questions Explored

Reflection Question

Question 1: What happens to the gradient as you approach the minimum?

As we approach the minimum, the gradient behavior reveals the elegance of gradient descent:

Results Analysis

Detailed Analysis of Gradient Behavior

1. **Automatic Deceleration:** The gradient approaches zero, naturally slowing our descent. This isn't programmed—it emerges from the mathematics.
2. **Precision Increase:** As gradients shrink, step sizes decrease, allowing precise convergence to the minimum.
3. **Mathematical Elegance:** For $f(x) = (x-3)^2 + 1$, we have $f'(x) = 2(x-3)$. As $x \rightarrow 3$, automatically $f'(x) \rightarrow 0$.
4. **Exponential Pattern:** The gradient doesn't decrease linearly—it follows an exponential decay, visible in the log-scale plot.

```

1 # Let's examine gradient behavior in detail
2 target = 3.0
3 distances = [1.0, 0.1, 0.01, 0.001, 0.0001]
4
5 print("Distance from Minimum | Gradient Value | Step Size (alpha=0.1)")
6 print("-" * 60)
7 for dist in distances:
8     x_point = target + dist
9     grad_val = gradient(x_point)

```

```
10 step_size = 0.1 * abs(grad_val)
11 print(f"{dist:20.6f}|_{grad_val:13.6f}|_{step_size:15.6f}")
12
13 # This shows how step sizes naturally shrink as we approach the minimum
```

Listing 5: Analyzing Gradient Behavior Near the Minimum

Reflection Question

Question 2: How does the learning rate affect convergence?

The learning rate fundamentally controls the trade-off between speed and stability:

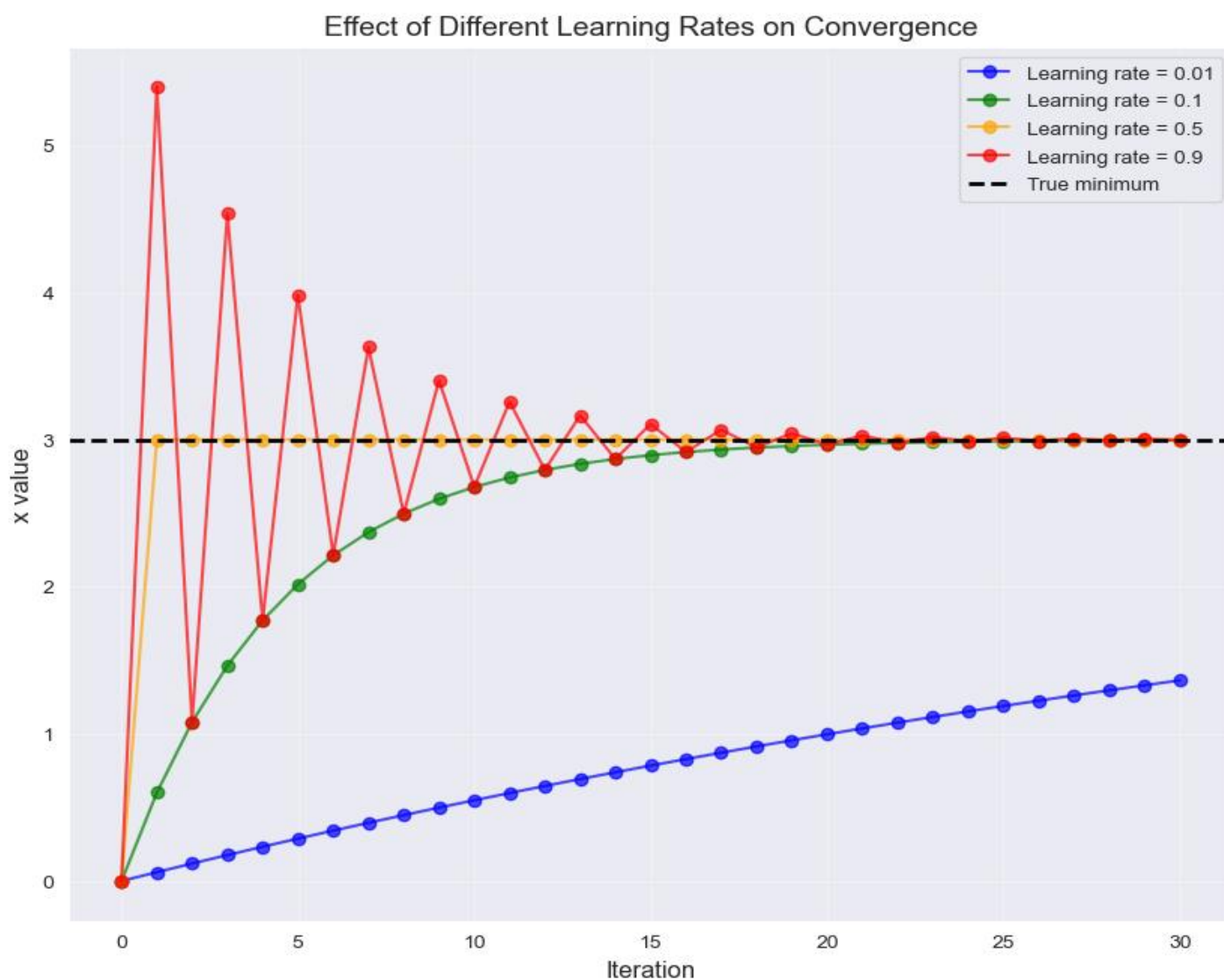


Figure 3: Impact of different learning rates on convergence patterns. Blue (0.01): Slow but stable. Green (0.1): Optimal balance. Orange (0.5): Faster with mild oscillation. Red (0.9): Dramatic oscillation but eventual convergence.

Results Analysis

Comprehensive Learning Rate Analysis

1. Learning Rate = 0.01 (Conservative)

- Behavior: Slow, monotonic convergence
- Advantage: Guaranteed stability
- Disadvantage: Requires many iterations
- Best for: Sensitive or ill-conditioned problems

2. Learning Rate = 0.1 (Optimal)

- Behavior: Smooth, efficient convergence
- Advantage: Good balance of speed and stability
- Disadvantage: None for this problem
- Best for: Well-behaved optimization landscapes

3. Learning Rate = 0.5 (Aggressive)

- Behavior: Faster initially, some overshooting
- Advantage: Quicker convergence
- Disadvantage: Less smooth path
- Best for: When computational budget is limited

4. Learning Rate = 0.9 (At the Edge)

- Behavior: Dramatic oscillation around minimum
- Advantage: Still converges eventually
- Disadvantage: Inefficient zig-zagging
- Risk: Approaching instability threshold

5. Learning Rate ≥ 1.0 (Critical)

- Behavior: Potential divergence
- Risk: Algorithm may fail to converge
- Reason: Steps become larger than the function's curvature can handle

Reflection Question

Question 3: What happens if we start further away from the minimum?

Starting position affects efficiency but not the fundamental behavior:

Results Analysis

Impact of Starting Position

1. Immediate Effects

- Longer journey to the minimum
- Larger initial gradients and step sizes
- Higher initial cost
- More iterations required

2. Algorithmic Robustness

- Gradient descent finds the same minimum regardless of starting point
- Convergence pattern remains exponential once near the minimum
- The algorithm naturally handles any reasonable starting position

3. Practical Insights

- In machine learning, we rarely know where optimal parameters lie
- Gradient descent's robustness to starting position is crucial
- Random initialization is a valid strategy because the algorithm will converge

```

1 # Test different starting positions
2 starting_positions = [-5, -2, 0, 5, 10]
3 learning_rate = 0.1
4 num_iterations = 30
5
6 print("Starting_Position | Iterations to 99% Accuracy | Final Error")
7 print("-" * 65)
8
9 for start_pos in starting_positions:
10     x_vals, _, _, _ = gradient_descent(start_pos, learning_rate,
11                                     num_iterations)
12
13     # Find when we get within 1% of the true minimum
14     for i, x in enumerate(x_vals):
15         if abs(3 - x) < 0.03: # Within 1% of minimum
16             iterations_needed = i
17             break
18     else:
19         iterations_needed = num_iterations
20
21     final_error = abs(3 - x_vals[-1])
22     print(f"{start_pos:16.1f} | {iterations_needed:24d} | {final_error:12.8f}")

```

Listing 6: Analyzing Starting Position Impact

6 Advanced Insights and Real-World Connections

Key Insight

Connecting to Modern Machine Learning

The principles we've explored scale to the cutting edge of AI:

1. **Neural Networks:** Each layer has thousands of parameters, but the same gradient-based optimization applies
2. **Deep Learning:** The "backpropagation" algorithm is essentially gradient computation for complex functions
3. **Large Language Models:** Models like GPT use variants of gradient descent to optimize billions of parameters
4. **Adaptive Methods:** Modern optimizers (Adam, RMSprop) build on gradient descent with sophisticated learning rate adaptation

7 Extended Exercises and Explorations

1. **Custom Cost Functions:** Modify the cost function to include multiple local minima and observe how gradient descent behaves
2. **Momentum Implementation:** Add momentum to accelerate convergence, especially when the cost function has elongated valleys
3. **Adaptive Learning Rates:** Implement learning rate schedules that decrease over time
4. **Noisy Gradients:** Add random noise to simulate stochastic gradient descent
5. **2D Extensions:** Expand to two dimensions and visualize gradient fields

```
1 # Framework for experimenting with variations
2 def custom_cost_function(x):
3     """Template for creating your own cost functions"""
4     # Example: Multiple local minima
5     return 0.1 * x**4 - 0.5 * x**3 - 2 * x**2 + 5 * x + 10
6
7 def gradient_descent_with_momentum(start, learning_rate, momentum,
8     iterations):
9     """Template for implementing momentum"""
10    velocity = 0
11    current_x = start
12
13    for i in range(iterations):
14        grad = gradient(current_x)
15        velocity = momentum * velocity - learning_rate * grad
16        current_x += velocity
17        # Track and visualize...
18
19    return current_x
```

```
19  
20 # Experiment with these and observe the differences!
```

Listing 7: Skeleton for Extended Explorations

8 Summary: The Elegance of Gradient Descent

Concept

Why Gradient Descent Works So Well

Gradient descent succeeds because it combines:

1. **Simplicity:** Only requires function value and gradient
2. **Universality:** Works for any differentiable function
3. **Scalability:** Extends to millions of parameters
4. **Robustness:** Handles various starting conditions
5. **Self-Regulation:** Naturally slows near optima

9 Final Reflection

Understanding gradient descent deeply provides the foundation for grasping modern machine learning. Every sophisticated optimization algorithm builds upon these fundamental principles. By mastering the simple case, you've developed intuition that applies to the most advanced AI systems being developed today.

Remember: Behind every breakthrough in computer vision, natural language processing, and reinforcement learning, gradient descent (or its variants) is quietly finding the optimal parameters, one step at a time, following the inexorable logic of calculus to minimize error and maximize performance.